

个人博客搜索功能实现指南

需求分析

用户希望在纯静态个人博客中添加搜索功能，要求：

- 实时筛选包含关键词的内容
- 模糊匹配，不区分大小写
- 搜索结果高亮显示
- 无匹配时显示提示信息
- 搜索时自动展开目录，空搜索时保持折叠状态

实现步骤

1. 添加搜索框样式

首先在 CSS 中添加搜索框、高亮和无结果提示的样式：

Code block

```
1  /* 搜索框样式 */
2  .search-container {
3      margin-bottom: 20px;
4      position: relative;
5  }
6
7  .search-input {
8      width: 100%;
9      padding: 12px 16px;
10     border: 2px solid #f0f0f0;
11     border-radius: 8px;
12     font-size: 14px;
13     transition: all 0.3s ease;
14 }
15
16 .search-input:focus {
17     outline: none;
18     border-color: #667eea;
19     box-shadow: 0 0 0 3px rgba(102, 126, 234, 0.1);
20 }
```

```
21
22  /* 高亮样式 */
23  .highlight {
24      background-color: #fff3cd;
25      color: #856404;
26      padding: 0 2px;
27      border-radius: 2px;
28      font-weight: 500;
29  }
30
31  /* 无结果提示 */
32  .no-results {
33      padding: 40px;
34      text-align: center;
35      color: #666;
36      background: #f9f9f9;
37      border-radius: 8px;
38      margin: 20px 0;
39  }
40
41  .no-results-icon {
42      font-size: 48px;
43      margin-bottom: 16px;
44      color: #ccc;
45  }
```

2. 添加搜索框HTML结构

在文档列表区域添加搜索框：

Code block

```
1  <section class="file-section">
2      <h2>📁 我的文档</h2>
3      <div class="search-container">
4          <input type="text" class="search-input" id="searchInput"
5              placeholder="搜索文件或文件夹...">
6      </div>
7      <div id="fileList">
8          <div class="loading">正在扫描文件目录</div>
9      </div>
10 </section>
```

3. 实现搜索逻辑

3.1 保存原始数据

添加一个变量来保存原始目录树数据，以便在搜索时使用：

Code block

```
1 let originalDirectoryTree = [];
```

3.2 添加搜索事件监听

在页面加载完成后，添加搜索输入事件监听：

Code block

```
1 // 添加搜索事件监听
2 searchInput.addEventListener('input', function(e) {
3     const searchTerm = e.target.value.trim().toLowerCase();
4     if (searchTerm === '') {
5         renderDirectoryTree(originalDirectoryTree);
6     } else {
7         const searchResults = searchInDirectory(originalDirectoryTree,
8 searchTerm);
9         renderDirectoryTree(searchResults);
10    }
11 });
```

3.3 实现搜索函数

创建 `searchInDirectory` 函数来递归搜索目录：

Code block

```
1 // 搜索目录
2 function searchInDirectory(items, searchTerm) {
3     const results = [];
4
5     items.forEach(item => {
6         if (item.type === 'directory') {
7             // 检查目录名是否匹配
8             if (item.name.toLowerCase().includes(searchTerm)) {
9                 // 如果目录名匹配，包含整个目录
10                results.push(item);
11            } else {
12                // 否则搜索子目录
13                const matchingChildren = searchInDirectory(item.children,
14 searchTerm);
```

```

14         if (matchingChildren.length > 0) {
15             // 如果子目录有匹配项，保留该目录
16             const directoryWithMatches = {...item, children:
matchingChildren};
17             results.push(directoryWithMatches);
18         }
19     }
20     } else if (item.type === 'file') {
21         // 检查文件名是否匹配
22         if (item.name.toLowerCase().includes(searchTerm)) {
23             results.push(item);
24         }
25     }
26 });
27
28 return results;
29 }

```

3.4 实现渲染函数

创建 `renderDirectoryTree` 函数来渲染搜索结果：

Code block

```

1  // 渲染目录树
2  function renderDirectoryTree(directoryTree) {
3      const fileListElement = document.getElementById('fileList');
4      const searchInput = document.getElementById('searchInput');
5      const searchTerm = searchInput ? searchInput.value.trim() : '';
6
7      if (directoryTree.length === 0) {
8          fileListElement.innerHTML = `
9              <div class="no-results">
10                  <div class="no-results-icon"><img alt="no results icon" /></div>
11                  <h3>未找到相关内容</h3>
12                  <p>请尝试使用其他关键词搜索</p>
13              </div>
14          `;
15          return;
16      }
17
18      fileListElement.innerHTML = '';
19      const treeElement = buildDirectoryTree(directoryTree, 'files',
!!searchTerm);
20      fileListElement.appendChild(treeElement);
21  }

```

3.5 修改目录树构造函数

修改 `buildDirectoryTree` 函数，添加自动展开功能和高亮显示：

Code block

```
1  // 构建目录树
2  function buildDirectoryTree(items, basePath, autoExpand = false) {
3      const ul = document.createElement('ul');
4      ul.className = 'directory-tree';
5
6      items.forEach(item => {
7          const li = document.createElement('li');
8          li.className = `tree-item ${item.type}`;
9
10         const itemContent = document.createElement('div');
11         itemContent.className = 'item-content';
12
13         if (item.type === 'directory') {
14             // 目录项
15             const toggle = document.createElement('span');
16             toggle.className = 'toggle';
17             toggle.textContent = '▶';
18             itemContent.appendChild(toggle);
19
20             const icon = document.createElement('span');
21             icon.className = 'icon';
22             icon.textContent = '📁';
23             itemContent.appendChild(icon);
24
25             const name = document.createElement('span');
26             name.innerHTML = highlightText(item.name);
27             itemContent.appendChild(name);
28
29             li.appendChild(itemContent);
30
31             // 子目录
32             const childrenUl = buildDirectoryTree(item.children, item.path,
autoExpand);
33             childrenUl.className = 'tree-children';
34             li.appendChild(childrenUl);
35
36             // 自动展开目录（仅在搜索时）
37             if (childrenUl.children.length > 0) {
38                 if (autoExpand) {
39                     toggle.classList.add('open');
40                     childrenUl.classList.add('open');
41                 }
            }
        }
    });
}
```

```

42
43         // 为目录添加点击事件
44         itemContent.addEventListener('click', (e) => {
45             e.stopPropagation();
46             toggle.classList.toggle('open');
47             childrenUl.classList.toggle('open');
48         });
49     } else {
50         // 如果没有子目录, 隐藏展开图标
51         toggle.style.display = 'none';
52     }
53 } else {
54     // 文件项
55     const icon = document.createElement('span');
56     icon.className = 'icon';
57     icon.textContent = getFileIcon(item.extension);
58     itemContent.appendChild(icon);
59
60     const link = document.createElement('a');
61     link.className = 'file-link';
62     link.href = item.path;
63     link.target = '_blank';
64     link.innerHTML = highlightText(item.name);
65     itemContent.appendChild(link);
66
67     // 添加文件类型数据属性, 用于样式应用
68     li.setAttribute('data-type', item.extension);
69
70     li.appendChild(itemContent);
71 }
72
73 ul.appendChild(li);
74 });
75
76 return ul;
77 }

```

3.6 实现高亮函数

创建 `highlightText` 函数来高亮显示搜索关键词:

Code block

```

1  // 高亮文本
2  function highlightText(text) {
3      const searchInput = document.getElementById('searchInput');
4      const searchTerm = searchInput ? searchInput.value.trim() : '';

```

```
5
6     if (!searchTerm) {
7         return text;
8     }
9
10    const regex = new RegExp(`${searchTerm}`, 'gi');
11    return text.replace(regex, '<span class="highlight">$1</span>');
12 }
```

核心功能解析

1. 模糊搜索

- 使用 `includes` 方法检查文件名或目录名是否包含搜索词
- 转换为小写进行比较，实现不区分大小写的搜索

2. 实时筛选

- 监听输入框的 `input` 事件，实时响应用户输入
- 每次输入时重新搜索并渲染结果

3. 高亮显示

- 使用正则表达式匹配搜索词
- 将匹配部分用 `<span class="highlight">` 包裹，应用高亮样式

4. 自动展开目录

- 搜索时自动展开所有包含匹配内容的目录
- 空搜索时保持目录折叠状态

5. 无结果提示

- 当搜索结果为空时，显示友好的提示信息

测试方法

1. 启动本地服务器：

Code block

```
1 python3 -m http.server 8000
```

2. 打开浏览器：访问 `http://localhost:8000`

3. 测试搜索功能：

- 在搜索框中输入关键词（如 "Modifier"）
- 观察搜索结果是否自动展开并高亮显示
- 清空搜索框，观察目录是否恢复折叠状态
- 输入不存在的关键词，观察无结果提示

4. 测试不同情况：

- 输入完整文件名
- 输入部分文件名
- 输入目录名
- 输入不同大小写的关键词

部署说明

该搜索功能完全基于前端实现，无需后端支持：

- 可以直接部署到 Netlify 或 GitHub Pages
- 不依赖任何外部库，仅使用 HTML、CSS 和 JavaScript
- 兼容所有现代浏览器

总结

通过以上步骤，我们成功为个人博客添加了一个功能完整、用户体验良好的搜索功能。该功能支持：

- 实时模糊搜索
- 不区分大小写
- 搜索结果高亮
- 自动展开目录
- 无结果提示

（注：文档部分内容可能由 AI 生成）